

Module 3: Software Design Engineering & UML 2.5 Modeling

Topics 27 to 39 • Modularity, Cohesion, Coupling, SOLID Principles, Use Case, Class, Sequence & Activity Diagrams

Author: Shaswat Raj (BIT Mesra) Academic Year: 2024–25 (5th Sem CSE) Standard: IEEE & ISO/IEC Standards

Topic 14: Software Design Principles & Architectural Engineering

Software Design bridges the gap between the requirements domain and the implementation domain. It defines the internal organization of software components, interfaces, and data structures.

Fundamental Design Concepts

- **Abstraction:** Hiding internal implementation complexity while exposing a clean, essential interface (e.g., procedural and data abstraction).
- **Modularity:** Decomposing a complex monolithic system into distinct, independently manageable components.
- **Information Hiding (David Parnas):** Encapsulating design decisions inside modules so that internal changes do not ripple across external callers.
- **Functional Independence:** Designing modules with high cohesion and low coupling.
- **Refinement:** Stepwise top-down elaboration from high-level abstract logic down to concrete algorithms.

Topic 15: Cohesion Taxonomy (7 Hierarchical Levels)

Cohesion measures the degree of functional relatedness among elements inside a single module. *High cohesion is the hallmark of superior software design!*

Cohesion Level	Quality Rating	Description	Concrete Code Example
1. Coincidental	Worst (0/10)	Elements grouped purely at random with zero logical relationship.	<code>`Utilities.java`</code> containing <code>`calculateTax()`</code> , <code>`printString()`</code> , <code>`sortArray()`</code> .
2. Logical	Poor (2/10)	Elements perform logically similar functions selected via a control flag.	<code>`handleInput(int flag)`</code> where flag 1=Mouse, flag 2=Keyboard, flag 3=Network.
3. Temporal	Fair (4/10)	Elements executed together during the same timeframe.	<code>`systemStartup()`</code> initializing DB connections, loading config files, and clearing cache.
4. Procedural	Moderate (5/10)	Elements executed in a specific sequence to accomplish a broader task.	<code>`processOrder()`</code> : <code>`checkStock()`</code> → <code>`deductBalance()`</code> → <code>`printReceipt()`</code> .
5. Communicational	Good (7/10)	Elements operate upon the exact same shared input dataset.	<code>`analyzeStudentRecords()`</code> : <code>`computeGPA(student)`</code> and <code>`generateTranscript(student)`</code> .
6. Sequential	Very Good (8.5/10)	Output of one element serves as the direct input to the next element (pipeline).	<code>`parseXML(file)`</code> → <code>`validateSchema(xml)`</code> → <code>`insertToDB(records)`</code> .
7. Functional	Best (10/10)	Every element contributes strictly to executing exactly one well-defined mathematical or business task.	<code>`Math.sin(angle)`</code> , <code>`calculateCompoundInterest(P, r, n, t)`</code> .

Topic 16: Coupling Taxonomy (6 Hierarchical Levels)

Coupling measures the degree of interdependence between separate software modules. *Low coupling minimizes ripple effects during system maintenance!*

Coupling Level	Quality Rating	Description	Concrete Architecture Example
1. Content	Worst (High)	One module directly accesses and modifies internal code or private data of another module.	Module A branches into the middle of Module B via <code>`goto`</code> or directly mutates B's private pointers.
2. Common	Very Poor	Multiple modules read and write to shared global variables or shared memory pools.	Modules A, B, and C all mutating global variable <code>`int g_system_state;`</code> .
3. Control	Poor	One module passes a control flag that dictates the internal execution path of another module.	<code>`calculatePayroll(employee, boolean isExecutive)`</code> .
4. Stamp (Data-Structured)	Fair / Good	Modules pass a complete composite data structure when only a few fields are needed.	Passing complete <code>`Customer`</code> object (30 fields) to a function that only needs <code>`customer.zipCode`</code> .
5. Data	Very Good	Modules communicate strictly by passing atomic primitive parameters through interfaces.	<code>`computeTax(double salary, double taxRate)`</code> .
6. Message / No Coupling	Best (Lowest)	Modules communicate asynchronously via message queues with zero shared state.	Microservices communicating via RabbitMQ JSON events.

Topic 17: Object-Oriented Design & The SOLID Principles

Principle	Core Definition	Violation vs. Clean Architectural Solution
S — Single Responsibility (SRP)	A class should have one, and only one, reason to change.	<i>Violation:</i> <code>`User`</code> class handling business logic + database saving + PDF email formatting. <i>Solution:</i> Split into <code>`User`</code> , <code>`UserRepository`</code> , and <code>`UserInvoicePdfService`</code> .
O — Open/Closed (OCP)	Software entities should be open for extension, but closed for modification.	<i>Violation:</i> Using huge <code>`switch(shapeType)`</code> statements to compute areas. <i>Solution:</i> Define abstract <code>`Shape`</code> interface with polymorphic <code>`calculateArea()`</code> implementations (<code>`Circle`</code> , <code>`Rectangle`</code>).
L — Liskov Substitution (LSP)	Subtypes must be substitutable for their base types without altering correctness.	<i>Violation:</i> <code>`Square`</code> inheriting from <code>`Rectangle`</code> where mutating width unexpectedly changes height. <i>Solution:</i> Extract common <code>`Shape`</code> interface; do not force square-rectangle inheritance.
I — Interface Segregation (ISP)	Clients should not be forced to depend upon interfaces they do not use.	<i>Violation:</i> Massive <code>`IMachine`</code> with <code>`print()`</code> , <code>`scan()`</code> , <code>`fax()`</code> , <code>`staple()`</code> . <i>Solution:</i> Segregate into <code>`IPrinter`</code> , <code>`IScanner`</code> , <code>`IFax`</code> .
D — Dependency Inversion (DIP)	High-level modules should not depend on low-level modules; both should depend on abstractions.	<i>Violation:</i> <code>`OrderService`</code> directly instantiating <code>`MySQLDatabase`</code> <code>db = new MySQLDatabase();</code> . <i>Solution:</i> Inject interface <code>`IDatabase`</code> via Dependency Injection.

Topic 18: UML 2.5 Modeling Suite (Structural & Behavioral Diagrams)

The **Unified Modeling Language (UML)** is the standard graphical language for specifying, visualizing, constructing, and documenting software system artifacts.

1. UML Use Case Diagram

- **Actor:** Stick figure representing external entity (User, External Payment Gateway).
- **Use Case:** Horizontal oval representing a distinct user task (``Place Order``).
- **Relationship ``<>``:** Mandatory dependency where the base use case unconditionally executes the included use case (``Place Order`` ``$>\rightarrow{\text{`<>`}}$`` ``Authenticate User``).

- **Relationship** `<>`: Optional behavior executed conditionally under specific extension points ('Place Order' $\xrightarrow{\text{<>}}$ 'Apply Discount Coupon').

2. UML Class Diagram

- **Class Structure**: 3-compartment rectangle (Class Name, Attributes, Operations).
- **Visibility Notations**: `+` (Public), `-` (Private), `#` (Protected), `~` (Package/Default).
- **Relationships**:
 - **Association**: Solid line representing structural link.
 - **Aggregation (White Diamond ◇)**: "Has-A" relationship where child can exist independently of parent (e.g., 'Department' ◇— 'Teacher').
 - **Composition (Black Diamond ◆)**: Strong "Part-Of" relationship with coincident lifecycles (e.g., 'Car' ◆— 'Engine').
 - **Generalization (Open Triangle △)**: "Is-A" inheritance relationship ('Dog' △— 'Animal').
 - **Realization (Dashed Arrow --> △)**: Implementing an Interface.

3. UML Sequence Diagram

- **Lifeline**: Vertical dashed line representing the existence of an object over time.
- **Activation Bar**: Narrow vertical rectangle indicating when object is executing code.
- **Messages**: Synchronous call (solid arrowhead $\rightarrow$), Asynchronous call (stick arrowhead $\rightarrow>$), Return message (dashed line $<--$).
- **Interaction Frames**: `alt` (if-else conditional branch), `opt` (optional single branch), `loop` (for/while loop iteration).

Topic 19: University Exam Problem Bank & Model Answers

Q6. Explain the Golden Rule of Software Design: "Maximize Cohesion, Minimize Coupling". (8 Marks)

1. **High Cohesion**: Ensures all elements in a module focus strictly on one unified responsibility. It makes the module easy to understand, test independently, and reuse across projects.
 2. **Low Coupling**: Minimizes dependencies between different modules. Changes inside module A's implementation details do not ripple across module B, dramatically reducing bug regression rates and maintenance costs.
- Summary*: High Cohesion + Low Coupling = High Modularity, Fault Isolation, and Maintainability!

Q7. Differentiate between Aggregation and Composition in UML Class Modeling with Concrete Examples. (8 Marks)

Dimension	Aggregation (Shared Association)	Composition (Composite Association)
UML Symbol	Hollow / White Diamond (◇) at whole end.	Filled / Black Solid Diamond (◆) at whole end.
Ownership	Weak ownership ("Has-A"). Parts can belong to multiple wholes.	Strong, exclusive ownership ("Part-Of"). Child belongs to exactly one parent.
Lifecycle Binding	Independent. If whole is destroyed, parts continue to exist.	Coincident. If whole is destroyed, all child components are automatically deleted.
Real-World Example	'University' ◇— 'Professor' (If university closes, professor exists).	'House' ◆— 'Room' (If house is demolished, rooms cease to exist).

Topic 38: Software Architectural Patterns & Enterprise Styles

Architectural Style	Structural Organization	Key Strengths	Limitations
Layered (N-Tier)	Hierarchical layers (Presentation → Business Logic → Data Access → Database).	Clear separation of concerns; easy to test layers in isolation.	Performance overhead due to cascading calls through every intermediate layer.
Pipe and Filter	Sequential stream where filters process data and pass streams through pipes (e.g., Unix CLI, ETL).	High reusability; filters can execute concurrently in parallel threads.	Difficult to handle interactive event-driven UI workflows.
Event-Driven (Pub-Sub)	Decoupled producers publish event messages to an event bus (Kafka); consumers subscribe.	Massive scalability; extreme decoupling between producers and consumers.	Eventual consistency challenges; difficult to trace distributed execution flow.
Microservices	Suite of independently deployable small services structured around business domains.	Independent horizontal scaling; polyglot technology stacks.	Complex distributed networking, network latency, distributed transaction rollbacks (Saga pattern).

Topic 39: Gang of Four (GoF) Software Design Patterns

Pattern Family	Classic Design Pattern	Architectural Purpose & Implementation Mechanism
Creational	Singleton Pattern	Guarantees a class has strictly one global instance with private constructor and synchronized <code>getInstance()</code> .
Creational	Factory Method	Defines an interface for creating an object, but lets subclasses decide which concrete class to instantiate.
Structural	Adapter Pattern	Converts the interface of a class into another interface clients expect, bridging incompatible interfaces.
Structural	Facade Pattern	Provides a unified, simplified high-level interface to a complex subsystem of dozens of internal classes.
Behavioral	Observer Pattern	Defines a one-to-many dependency where state changes in a Subject automatically notify all registered Observers.
Behavioral	Strategy Pattern	Encapsulates interchangeable algorithmic families inside separate classes, switching algorithms dynamically at runtime.

Topic 40: University Exam Problem Bank (Part II)

Q16. Draw and Explain the Interaction Frames in a UML Sequence Diagram: alt, opt, loop, and par. (8 Marks)

Interaction frames represent complex control logic inside sequence diagrams:

- alt (Alternative):** Models mutually exclusive conditional execution (if-else). Separated by horizontal dashed divider lines with guard conditions `[[balance >= amount]]` and `[else]`.
- opt (Optional):** Executes child messages only if the single guard condition is true (equivalent to a simple `if` without `else`).
- loop (Iteration):** Repeats the message sequence for a specified count or while guard condition holds `[while hasNext()]`.
- par (Parallel):** Enclosed sub-fragments execute concurrently in parallel asynchronous threads!

Q17. Explain the Model-View-Controller (MVC) Architectural Pattern with Component Interactions. (8 Marks)

MVC segregates UI display from core business logic and state:

1. **Model:** Encapsulates business data state, database queries, and business rules. Completely independent of the UI.
2. **View:** Renders visual interface (HTML/CSS, mobile screens) based on Model state.
3. **Controller:** Listens to user inputs (mouse clicks, HTTP POST), translates them into business updates on the Model, and selects the appropriate View to render.

Topic 41: Object Constraint Language (OCL) in UML Modeling

OCL is a formal declarative specification language used within UML class diagrams to specify precise mathematical constraints and business rules without implementation code:

```
-- Invariant: Bank Account balance cannot drop below overdraft limit
context BankAccount
inv: self.balance ≥ -self.overdraftLimit

-- Pre-condition and Post-condition on withdrawal operation
context BankAccount::withdraw(amount: Real): Boolean
pre: amount > 0 and (self.balance - amount ≥ -self.overdraftLimit)
post: self.balance = self.balance@pre - amount and result = true
```

** Solved Architecture Case: Refactoring Legacy Code to Strategy Design Pattern**

Problem: A monolithic billing class uses complex nested conditionals: `if (type == "CREDIT") chargeCredit(); else if (type == "PAYPAL") chargePayPal(); ...` (Violates Open/Closed Principle!).

Refactoring Solution:

1. Define common interface: `public interface PaymentStrategy { void pay(double amount); }`.
2. Implement concrete classes: `CreditCardStrategy`, `PayPalStrategy`, `CryptoStrategy`.
3. Context class `ShoppingCart` accepts `PaymentStrategy` via constructor, achieving 100% decoupled extensibility!

Topic 42: Complete Gang of Four (GoF) Design Patterns Catalog

Family	Pattern Name	Intent & Structural Mechanism
Creational	Builder Pattern	Separates the construction of a complex object from its representation, allowing the same construction process to create different representations.
Creational	Prototype Pattern	Specifies the kind of objects to create using a prototypical instance, creating new objects by cloning this prototype.
Structural	Decorator Pattern	Attaches additional responsibilities to an object dynamically at runtime, providing a flexible alternative to subclassing.
Structural	Proxy Pattern	Provides a surrogate or placeholder for another object to control access to it (e.g., Virtual Proxy, Remote Proxy, Protection Proxy).
Behavioral	Command Pattern	Encapsulates a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support undoable operations.
Behavioral	State Pattern	Allows an object to alter its behavior when its internal state changes. The object will appear to change its class.
Behavioral	Template Method	Defines the skeleton of an algorithm in an operation, deferring some steps to subclasses without changing algorithm structure.

** Solved Architecture Case: Refactoring to Decorator Pattern for Coffee Billing**

Rather than creating 2^n subclasses (`EspressoWithCaramelAndMilk`), wrap base `Coffee` with `CaramelDecorator` and `MilkDecorator`, dynamically chaining `getCost()`: $5 + 2 + 1 = \$8!$

🏛️ Solved Problem 23: Cohesion & Coupling Metric Derivation (LCOM & CBO)

- **Lack of Cohesion in Methods (LCOM - Chidamber & Kemerer):** Let P be method pairs with disjoint attribute sets, Q be method pairs sharing attributes. $LCOM = \max(0, |P| - |Q|)$. High LCOM $\implies$ Poor cohesion (class should be split!).
- **Coupling Between Objects (CBO):** Count of external classes to which a class is coupled. High CBO $\implies$ High ripple effect vulnerability during refactoring!

🏛️ Solved Architecture Case: Observer Pattern in Real-Time Stock Market Dashboard

Subject `StockTicker` maintains `List`. When stock prices fluctuate, `notifyObservers()` updates `MobileAppView` and `LedgerService` concurrently with zero tight coupling!

🏛️ Solved Problem 32: UML State Machine Diagram with Composite States

Model an ATM Card Reader with composite state `Card Processing` containing substates `PIN Verification` and `Account Selection` with shallow history node (H) restoring previous active substate upon momentary power fluctuation!

🏛️ Solved Problem 33: UML Component & Deployment Diagram Architecture

- **Component Diagram:** Models physical software modules (JARs, DLLs, Docker images) with `<>` stereotypes and provided/required lollipop interfaces.
- **Deployment Diagram:** Models physical hardware compute nodes (Web Server, Database Server) and execution environments connected via communication paths (`TCP/IP`, `gRPC`).

🏛️ Solved Problem 42: SOLID Principles — Open/Closed Principle (OCP) Code Transformation

```
// Refactored Clean Architecture adhering strictly to Open/Closed Principle
public interface Shape { double calculateArea(); }

public class Circle implements Shape {
    private double radius;
    public Circle(double radius) { this.radius = radius; }
    public double calculateArea() { return Math.PI * radius * radius; }
}

public class AreaCalculator {
    public double totalArea(List shapes) {
        return shapes.stream().mapToDouble(Shape::calculateArea).sum();
    }
}
```

Verification: Adding a new `Triangle` shape requires zero edits to `AreaCalculator`, achieving 100% modification immunity!

🏛️ Solved Problem 43: Structural Decorator Pattern vs Subclassing Explosion

In a Graphical Windowing system: Rather than declaring 32 subclasses (`BorderedScrollableDarkWindow`), dynamically attach `BorderDecorator`, `ScrollDecorator`, and `ThemeDecorator` wrapping the base `Window` interface!

🏛️ Solved Problem 52: Factory Method vs Abstract Factory Pattern

- **Factory Method:** Uses inheritance and relies on a derived class to instantiate a single product object (`createButton()`).
- **Abstract Factory:** Uses object composition to create entire *families* of related or dependent objects without specifying their concrete classes (`GUIFactory` creates `Button`, `Checkbox`, `Scrollbar` together)!

🏛️ Solved Problem 53: Law of Demeter (Principle of Least Knowledge)

A method M of object O should only call methods of: (1) O itself, (2) M 's parameters, (3) objects created within M , (4) O 's direct member objects. Prohibits train wrecks: ``a.getB().getC().getD().doSomething()`` $\implies$ Refactor to ``a.doSomething()``!

🏛️ Solved Problem 62: Gang of Four Singleton Thread-Safe Double-Checked Locking

```
public class DatabaseConnectionPool {
    private static volatile DatabaseConnectionPool instance;
    private DatabaseConnectionPool() { /* Initialize pool */ }

    public static DatabaseConnectionPool getInstance() {
        if (instance == null) {
            synchronized (DatabaseConnectionPool.class) {
                if (instance == null) {
                    instance = new DatabaseConnectionPool();
                }
            }
        }
        return instance;
    }
}
```

🏛️ Solved Problem 63: Adapter Pattern for Legacy Payment Gateway Integration

Client expects ``ModernPayment`` interface (``pay(amount)``). Legacy gateway uses ``OldGateway`` (``makeTransaction(cents, currency)``). ``PaymentAdapter`` implements ``ModernPayment``, delegating and converting dollars to cents internally!

🏛️ Solved Problem 71: Structural Facade Pattern in Complex Subsystems

In a home theater system: ``HomeTheaterFacade`` exposes a single simple ``watchMovie()`` method that coordinates ``Projector.on()``, ``SoundSystem.setVolume()``, ``DvdPlayer.play()``, and ``Lights.dim()``, shielding external callers from 15 separate low-level interfaces!

🏛️ Solved Problem 72: Behavioral Command Pattern with Undo Mechanism

Each user action is an instance of ``Command`` with ``execute()`` and ``undo()`` methods stored on a history stack, enabling infinite multi-level undo/redo operations!

🏛️ Solved Problem 82: Creational Abstract Factory vs Builder

- **Abstract Factory:** Focuses on creating families of related objects all at once.
- **Builder:** Focuses on constructing a complex composite object step-by-step through a fluent API (``User.builder().name("Alice").age(30).build()``)!

🏛️ Solved Problem 83: Structural Proxy Pattern Types

(1) **Virtual Proxy** (defers loading huge images/DB records until needed), (2) **Protection Proxy** (checks user authorization permissions before invoking target object), (3) **Remote Proxy** (handles network serialization via gRPC/RMI)!

🏛️ Solved Problem 84: Behavioral Observer Memory Leak Prevention (Weak References)

When subjects hold strong references to observers, forgotten unregistered observers cause memory leaks (*Lapsed Listener Problem*). Fix by storing observers via ``WeakReference`` or automatic cleanup lifecycle hooks!

 Solved Problem 96: Structural Composite Pattern in Hierarchical File Systems

'FileSystemComponent' is implemented by leaf 'File' and composite 'Directory' containing 'List', treating individual files and nested folders uniformly!

 Solved Problem 97: Behavioral Chain of Responsibility Pattern

Decouples the sender of a request from its receivers by giving multiple handler objects ('AuthHandler' → 'RateLimitHandler' → 'ValidationHandler') a chance to handle the request!

 Solved Problem 98: Behavioral Mediator Pattern

Prevents an $O(n^2)$ web of direct object references by routing all dialog component interactions ('Button', 'TextBox', 'ListBox') through a central 'DialogMediator'!

 Solved Problem 110: Behavioral Visitor Pattern in Abstract Syntax Tree Processing

Separates algorithm operations ('TypeCheckerVisitor', 'CodeGeneratorVisitor') from complex heterogeneous object node hierarchies ('BinaryExpr', 'AssignStmt'), letting you add new compiler passes without modifying node classes!

 Solved Problem 111: Architectural Anti-Patterns & Code Smells

- **God Object (Blob):** Single monolithic class monopolizing all system logic $\implies$ Refactor into cohesive services.
- **Spaghetti Code:** Unstructured control flow with high coupling $\implies$ Refactor to layered architecture.
- **Lava Flow:** Dead legacy code nobody dares touch $\implies$ Eliminate using test-driven refactoring!

 Solved Problem 112: Domain-Driven Design (DDD) Bounded Contexts & Ubiquitous Language

Decomposes complex enterprise domains into distinct Bounded Contexts ('BillingContext', 'ShippingContext'), ensuring terms have unambiguous definitions within their domain boundaries!

 Solved Problem 123: Structural Bridge Pattern in Cross-Platform UI Toolkits

Decouples an abstraction ('Window') from its platform-specific implementation ('WinWindowImpl', 'MacWindowImpl', 'LinuxWindowImpl'), allowing both hierarchies to vary independently!

 Solved Problem 124: Behavioral Memento Pattern for Game State Checkpointing

Captures and externalizes an object's internal state without violating encapsulation, allowing the state to be restored later ('GameStateMemento')!

 Solved Problem 131: Behavioral Iterator Pattern & Fail-Fast vs Fail-Safe Iterators

- **Fail-Fast Iterator (Java 'ArrayList'):** Throws 'ConcurrentModificationException' immediately if collection is modified structurally during traversal.
- **Fail-Safe / Snapshot Iterator ('CopyOnWriteArrayList'):** Operates on a cloned snapshot of the array, permitting concurrent multi-threaded modifications with zero locking overhead!

 Solved Problem 132: Creational Flyweight Pattern for Large-Scale Text Rendering

In a text editor containing 1,000,000 characters: Rather than creating 1,000,000 separate 'Character' objects, share 26 intrinsic font glyph objects ('GlyphFactory') and pass coordinates externally, slashing RAM consumption from 80MB to 200KB!

 Solved Problem 133: Behavioral State Pattern vs Finite State Machine Enums

Encapsulates distinct state-specific behaviors inside polymorphic subclasses (`DraftState`, `ModerationState`, `PublishedState`), eliminating massive 500-line nested `switch-case` statements in Document publishing workflows!

 Solved Problem 140: Architectural Microservices Event Sourcing & CQRS

- **Event Sourcing:** Persists state mutations as an append-only log of immutable domain events (`OrderPlaced`, `PaymentReceived`).
- **CQRS:** Segregates Command (write) models from Query (read) models, scaling read performance independently by $100\times$!

 Solved Problem 145: Architectural CQRS Read/Write Replica Synchronization

Using distributed Kafka change data capture (CDC) streams to asynchronously replicate write model commits to Elasticsearch read indexes with sub-second lag!

 Solved Problem 146: Behavioral Strategy Pattern in Dynamic Routing Algorithms

Context `NavigationEngine` swaps `DijkstraShortestPath`, `AStraTrafficAware`, and `EcoFriendlyRoute` strategies at runtime based on real-time GPS telemetry!

 Solved Problem 153: Gang of Four Template Method Pattern in Data ETL

Abstract class `DataETLPipeline` defines `process()` executing `extract()`, `transform()`, and `load()`, while letting subclasses override `extract()` for SQL or NoSQL sources!

 Solved Problem 154: UML Package Diagram & Dependency Management

Modeling modular boundaries with `<>` and `<>` stereotypes to prevent cyclic package dependencies and enforce Clean Architecture layers!

 Solved Problem 161: Microservices Saga Pattern vs Two-Phase Commit (2PC)

- **2PC:** Synchronous blocking distributed transaction locking databases across network boundaries; fails under network partitions.
- **Saga Pattern:** Asynchronous choreography/orchestration executing local transactions with compensating rollback actions (`cancelOrder()`), guaranteeing eventual consistency without blocking locks!

 Solved Problem 162: Behavioral Visitor Pattern Implementation in Type Inference Engine

Polymorphically visiting syntax nodes (`VariableDecl`, `FunctionCall`, `BinaryOp`) to infer hindley-milner types with zero AST class pollution!

 Solved Problem 167: Enterprise API Gateway Architectural Pattern

Consolidates routing, SSL termination, rate limiting, and JWT authentication token validation into a single reverse-proxy endpoint (Kong / Spring Cloud Gateway), preventing client coupling to 50 downstream microservices!

**Solved Problem 168: Object-Oriented Design Metrics (Chidamber & Kemerer Suite)**

- **WMC (Weighted Methods per Class):** Measures class complexity.
- **DIT (Depth of Inheritance Tree):** Deep trees (> 5) increase fragility.
- **NOC (Number of Children):** High NOC indicates high reuse but severe change ripple risk!